

CS 610 GPU Programming (Winter 2026)

Project Final Report

Group No. 1

Members: Zac Cross

1. Introduction

- a. Introduce the problem/algorithm*
- b. Why it is important to the field?*
- c. What was the original design on CPUs?*

For my term project, I chose to accelerate my earthquake cycle simulation code by optimizing several core operations inside a Conjugate Gradient (CG) solver. In particular, I implemented a matrix-free derivative operator and accelerated several matrix–vector operations that appear inside the CG algorithm. Traditionally, earthquake modeling with numerical methods such as finite differences or finite elements follows a fairly standard workflow. First, the physical problem is set up using the governing equations for elasticity and fault friction. Then the system is evolved forward in time using a time stepping method such as Forward Euler or a Runge–Kutta scheme. The goal of these simulations is to observe how the Earth responds to different fault geometries, material parameters, and friction laws, and ultimately to improve our understanding of geophysical processes.

However, depending on the problem formulation and domain size, earthquake simulations quickly become computationally expensive. One major challenge is the presence of many competing spatial and temporal scales. For example, material parameters can vary on the scale of microns, while the domains of interest are often tens to hundreds of kilometers in size. Earthquake cycle simulations also involve extreme differences in time scales: rupture events may occur over minutes, while the time between events can span centuries or even millennia. Capturing these effects accurately requires very large computational domains and long simulation times, which makes performance a major concern.

In this project, I focus specifically on the physics computations that occur at each time step, since these operations are typically the main computational bottleneck in earthquake modeling. In my own research, I use a numerical method called Summation-By-Parts with Simultaneous Approximation Terms (SBP-SAT). This is a high-order finite difference method that mimics the integration-by-parts property from calculus while enforcing boundary conditions weakly through penalty terms. These properties give the method strong stability guarantees, which makes it an attractive foundation for complicated geophysical simulations.

When spatial derivatives are discretized using SBP-SAT operators, the main computation often reduces to solving a large linear system of the form

$$Ax = b$$

where A is the discretized differential operator, x is the unknown quantity of interest, and b represents forcing terms and boundary contributions.

For small domains this system can often be solved with a direct solver. However, as the problem size grows—as it must for realistic earthquake simulations—direct solves quickly become intractable in both memory and computational cost. Instead, most models rely on iterative solvers such as Jacobi, multigrid, or Krylov methods to progressively refine an initial guess until the solution converges within some tolerance.

In this project I chose to use a Conjugate Gradient (CG) solver because it has well-understood convergence properties and works efficiently for large sparse systems. One limitation of CG is that it requires the system matrix to be symmetric positive definite (SPD). However, SBP-SAT operators are naturally symmetric negative definite, so converting the system to SPD form is straightforward.

The CG algorithm proceeds by repeatedly computing a residual between the product of the operator A and the current solution estimate x and the right-hand side vector b and then updating the solution using a sequence of orthogonal search directions. At first glance CG appears to be a very serial algorithm, since each iteration depends on the results of the previous one. However, the underlying computations inside each iteration consist primarily of matrix–vector products, vector updates, and dot products. From what we learned in class this quarter, these operations are highly parallelizable.

In particular, the action of an SBP-SAT operator on a vector closely resembles a stencil operation with additional boundary terms. Because stencil computations map naturally to parallel hardware, this structure makes the CG solver a strong candidate for GPU acceleration.

Therefore, in this project I focus on optimizing the linear algebra kernels inside the CG solver, including the matrix-free derivative operator and several vector operations, in order to accelerate earthquake cycle simulations on GPU hardware.

2. Goal

- a. *What is the goal of the project?*
- b. *What potential impact it would make if the problem/algorithm is successfully accelerated using GPUs?*

The goal of this project is to accelerate the Conjugate Gradient (CG) solve for an SBP-SAT discretization of the 1D linear elasticity equation using GPU parallelism. The governing equation considered in this project is

$$\frac{\partial^2 u}{\partial t^2} = \mu \frac{\partial^2 u}{\partial x^2} + f$$

where u represents the material displacement, x is the spatial coordinate, t is time, μ is a material parameter called the shear modulus from Hooke's Law, and f is a forcing term.

To simplify the problem for this project, I focus on the static case, where the time derivative is zero. The governing equation then reduces to

$$0 = \mu \frac{\partial^2 u}{\partial x^2} + f.$$

Using the SBP-SAT discretization described earlier, the discrete system takes the form

$$0 = \mu D_{xx} u + SAT + f$$

on the spatial domain

$$x \in \Omega$$

The domain, boundary conditions, and SAT penalty terms will be specified in the next section.

The primary goal of this project is to accelerate the CG solver applied to this system. Since CG is repeatedly applied throughout a simulation, accelerating this solver can significantly reduce the total runtime of earthquake cycle models. If successful, this would enable simulations with larger domains and higher spatial resolution. More importantly, this work lays the groundwork for extending the method to higher dimensions. While the 1D problem is relatively small, the size of SBP operators grows rapidly in 2D and 3D, making explicit matrix storage impractical, even with sparse representations. A matrix-free implementation combined with GPU acceleration reduces memory requirements and enables the solution of significantly larger systems.

3. Approach

- a. *Which parts can be accelerated using GPUs?*
- b. *How to exploit the parallelism in the original problem/algorithm?*
- c. *What is the overall structure of your program?*
- d. *Design of each key kernels*
- e. *What optimization techniques were used?*

Following the description of CG above, I target the linear algebra operations inside the CG solver for GPU acceleration. I initially explored whether it would be possible to interleave CG operations using CUDA streams to overlap computation and data movement. However, the computational dependency graph of CG is almost entirely linear: each iteration depends on results from the previous iteration. Additionally, the required data transfers occur sequentially when entering and exiting the CG solver, which limits the benefit of techniques such as pinned memory and asynchronous memory copies. As a result, I focused primarily on accelerating the matrix and vector operations that dominate the cost of each CG iteration. Each CG iteration repeatedly performs these operations, making them the primary computational cost of the solver.

The overall structure of the program is as follows. First, the explicit SBP operators are constructed for orders 2, 4, or 6. Next, the SAT penalty terms are added to enforce the boundary conditions. After this, the computational domain and source terms are initialized. Finally, the CG solver is executed to compute the numerical solution, which is then compared against an exact solution. The setup of the exact solution using the method of manufactured solutions will be discussed later in the Evaluation section.

The GPU acceleration focuses on the linear algebra kernels inside the CG method. Three main operations were implemented on the GPU: vector addition, vector dot product, and matrix-vector product. Special emphasis was placed on developing a matrix-free implementation of the matrix-vector multiplication. The vector addition kernel was trivial to implement, so we do not go into detail in this report.

For all kernels described below, the same general execution strategy is used. The number of threads launched matches the number of elements in the output vector, and a 1D thread block layout is used to match the 1D structure of the data.

Vector Dot Product Kernel:

Next, I implemented the vector dot product kernel. This operation is more challenging to parallelize because it requires a reduction from a vector to a scalar.

In a naïve implementation, each thread computes a single multiplication, $c[tid]=a[tid]\times b[tid]$, and a single thread then reduces the entire vector to a scalar value. However, this approach performed significantly worse than the CPU implementation due to the serial reduction step.

To improve performance, I implemented a parallel reduction strategy. Within each thread block, a decreasing-stride reduction is used to combine partial sums in shared memory. After the block-level reduction is complete, the final results from each block are accumulated into the global result using atomic additions. This approach significantly improved the performance of the dot product operation compared to the naïve version.

Matrix–Vector Multiplication:

The matrix–vector multiplication kernel was the most complex and time-consuming part of the GPU implementation. In the initial implementation, each thread computed a dot product between one row of the matrix A and the vector x . As a first optimization attempt, I implemented a to reuse elements of the vector x across threads. However, this produced only limited performance improvements.

A potential improvement would be to assign multiple threads to each dot product and perform the reduction cooperatively within a thread block. However, since the final goal of the project was to implement a matrix-free operator, I instead focused on that approach.

In the matrix-free implementation of the matrix–vector product, the SBP-SAT stencil is hard-coded directly into the kernel. For the second-order operator, this corresponds to a three-point stencil applied to each output element. Because each stencil operation only depends on neighboring elements, this formulation maps naturally to GPU parallelism and avoids the need to explicitly store the full matrix.

To further optimize performance, I reduced the register usage per thread by fusing several arithmetic operations together. This helped increase occupancy and improved overall kernel performance.

4. Challenges

- a. What challenges you encountered during the project?*
- b. What were the solutions?*

Several challenges arose during the development of the GPU implementation. One of the first issues was ensuring correctness between the CPU and GPU implementations. Because floating-point arithmetic on GPUs can occur in a different order than on CPUs, small numerical differences between the two implementations are expected. These differences can accumulate slightly during iterative methods like Conjugate Gradient, resulting in solutions that are extremely close but not exactly identical. To address this, I compared solutions using a tolerance scaled with the problem size rather than expecting bitwise identical results. This allowed me to confirm that the GPU implementation was producing numerically consistent solutions while accounting for expected floating-point variation.

Another challenge involved debugging memory issues. Memory bugs can be particularly difficult to diagnose in GPU programs, especially when mixing host and device memory allocations. Additionally, compiling with CUDA support changed some aspects of how memory was allocated and managed on the heap, which occasionally introduced subtle issues that were not present in the CPU-only implementation. As a result, careful attention to memory allocation and deallocation was necessary throughout the project to avoid memory leaks and ensure stability.

A final challenge was determining the appropriate CUDA block size for the kernels. Since the data structures in this project are one-dimensional, both the grid and block configurations were implemented using 1D layouts. However, performance was sensitive to the choice of block size. Through experimentation, I found that a block size of 32 threads provided the best performance for the CG kernels on the hardware used in this project. Achieving good performance also required selecting configurations that made effective use of shared memory and avoided unnecessary register pressure. These parameters were tuned using a combination of trial-and-error testing and profiling with NVIDIA Nsight Compute.

5. Evaluation

- a. *Performance of key kernel (time, memory bandwidth, compute utilization, etc.)*
- b. *Performance of the whole program (comparing with the original design)*

Convergence Testing:

Before evaluating performance, I verified the correctness of both the dense and matrix-free operators using the **method of manufactured solutions (MMS)**. For this test, the manufactured solution

$$u(x, t) = t \sin(\pi x)$$

On the domain

$$x \in \Omega = [-10, 10].$$

For the MMS, we add the source term

$$f = t \pi^2 \sin(\pi x).$$

Finally, for our SAT term, we enforce Dirichlet or displacement conditions, enforcing conditions on $x = -10, 10$. By using the MMS, the exact solution to be known analytically so the numerical error can be measured directly.

For the dense SBP-SAT operators, convergence tests were performed for orders $p = 2, 4, 6$. In each case, the numerical solution demonstrated the expected order of convergence as the grid was refined, confirming that the discretization and CG solver were implemented correctly. These tests provide confidence that the dense operator accurately represents the SBP-SAT discretization.

The same MMS setup was also used to verify the matrix-free implementation, though only the $p = 2$ operator was implemented and tested in this case for the sake of time. Because the matrix-free kernel directly applies the stencil rather than explicitly storing the matrix, confirming that the solution converges at the expected rate was an important validation step. The matrix-free implementation produced the same convergence behavior as the dense version, confirming that the stencil-based kernel correctly reproduces the SBP operator.

Linear Algebra Kernels:

To evaluate the performance of the GPU implementation, I compared the runtime of each linear algebra kernel against the equivalent CPU implementation across a range of problem sizes. Each test was run multiple times, and the average runtime was recorded. The kernels evaluated include vector addition, vector dot products, explicit matrix–vector multiplication, and the matrix-free stencil implementation of the SBP operator. For the plots reported below, I calculate speedup as:

$$\frac{\text{CPU Time (s)}}{\text{GPU Time (s)}}.$$

The vector dot product results show that the CPU implementation remained faster for all problem sizes tested. Even with the optimized parallel reduction, the GPU implementation did not outperform the CPU version for the problem sizes used in this project. This is largely due to the reduction step and the relatively small problem sizes, which limit the amount of parallel work available for the GPU. In practice, the dot product is also a relatively small portion of the total CG runtime compared to the matrix–vector multiplication.

The explicit matrix–vector multiplication showed much more significant improvements. For small matrices the speedup is modest, but as the system size increases the GPU implementation scales much better than the CPU version. At a grid size of 32,000, the GPU implementation achieved roughly 50× speedup compared to the CPU implementation. This improvement occurs because the matrix–vector multiplication exposes a large amount of parallel work, allowing the GPU to fully utilize its available threads.

The largest improvement was observed in the matrix-free stencil implementation of the SBP operator. Instead of explicitly storing the matrix and computing a full row-wise dot product, the matrix-

free implementation directly applies the finite difference stencil. This significantly reduces memory usage and improves memory locality. As a result, the GPU implementation achieved speedups ranging from roughly $10\times$ for smaller grids to over $90\times$ for the largest problem size tested (1.6 million grid points). This result highlights the main benefit of the matrix-free approach: eliminating the need to store the full operator both reduces memory overhead and exposes a highly parallel stencil computation that maps naturally to GPU hardware.

Overall, these results demonstrate that the most important performance gains come from accelerating the matrix–vector operations, particularly when combined with a matrix-free formulation. While simple vector operations provide limited benefit from GPU acceleration at small scales, the stencil-based matrix-free operator shows substantial speedups and enables the simulation of much larger systems. These improvements are particularly important for extending the method to higher dimensional problems, where explicitly storing SBP operators becomes prohibitively expensive.

Plots of speedups for all the above kernels are shown below at the end of the report.

Conjugate Gradient:

To evaluate the overall impact of these kernel optimizations, I also measured the total runtime of the Conjugate Gradient solver using both the dense matrix formulation and the matrix-free stencil implementation. Unlike the individual kernel benchmarks above, these measurements include all data transfers required to execute the solver. This provides a more realistic comparison to how the algorithm would perform in a plug-and-play simulation environment.

For the dense formulation of the operator, the GPU implementation shows clear performance improvements as the system size increases. For very small grid sizes the CPU implementation remains faster because the GPU kernel launch and transfer overhead dominate the runtime. However, once the grid size exceeds roughly 400 grid points, the GPU solver begins to outperform the CPU version. At the largest problem tested (grid size 3200), the optimized GPU CG implementation achieves approximately $7.5\times$ speedup compared to the CPU implementation. This result reflects the benefit of accelerating the matrix–vector operations and reductions inside the CG loop, which dominate the cost of the dense solve.

The matrix-free CG implementation shows a slightly different performance profile. Because the stencil operator itself is extremely lightweight, the relative cost of data transfers becomes more noticeable for small problems. As a result, the CPU solver remains faster for smaller grid sizes. However, once the problem size grows beyond approximately 8000 grid points, the optimized GPU implementation begins to outperform the CPU version. At 16,000 grid points the GPU solver achieves roughly $4\times$ speedup, and continues to scale well for larger systems. These results demonstrate that while data transfer overhead can limit performance for smaller problems, the combination of matrix-free operators and GPU acceleration becomes increasingly beneficial as the problem size grows.

Overall, these results reinforce the conclusions from the kernel benchmarks: the greatest benefit from GPU acceleration occurs when the solver performs large numbers of highly parallel stencil operations. As the dimensionality of the problem increases to 2D and 3D, both the problem size and the cost of the derivative operators grow significantly. In those regimes, the matrix-free GPU implementation presented here is expected to provide even larger performance benefits.

Plots of speedups for all the above kernels are shown below at the end of the report.

It's important to note that I only tested Time to Execute for all the kernels and implementations. Further work could be done to measure occupancy, memory bandwidth, and arithmetic intensity to further improve kernel performance.

Figures of Speedup for Linear Algebra:

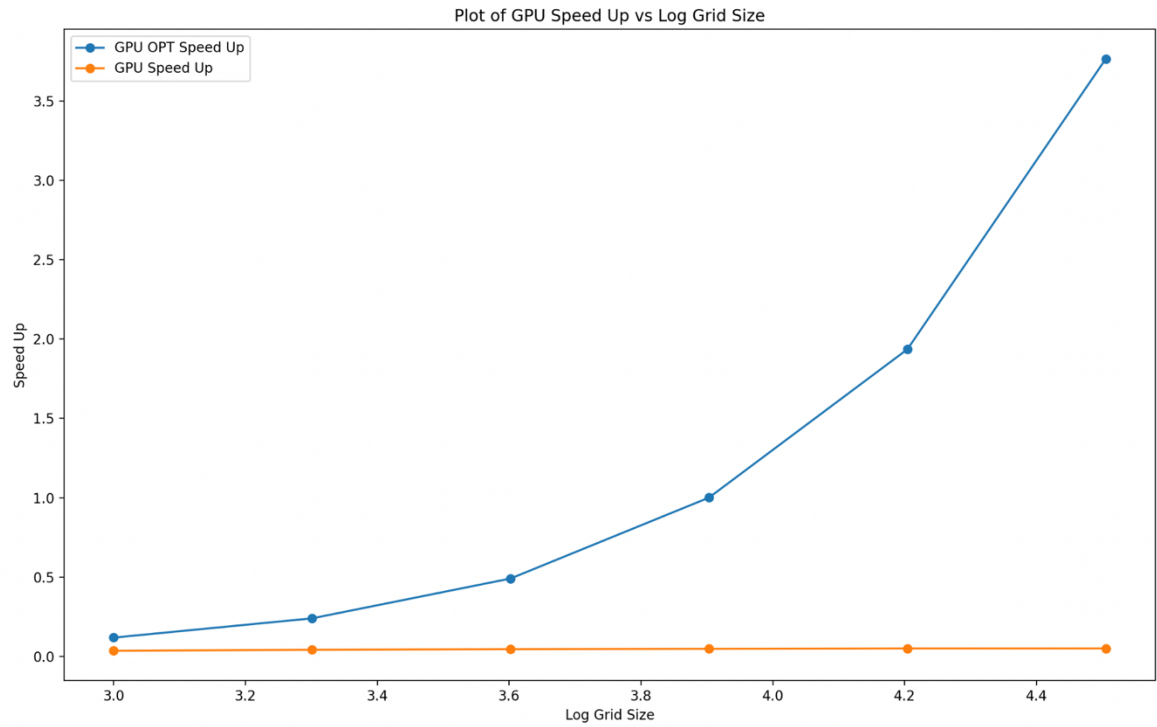


Figure 1: Plot of GPU Speedup vs Log Grid Size for optimized and non-optimized code for Vector Dot Product. We see the optimized code outperforms the CPU for larger grid sizes, but the naïve code does significantly worse.

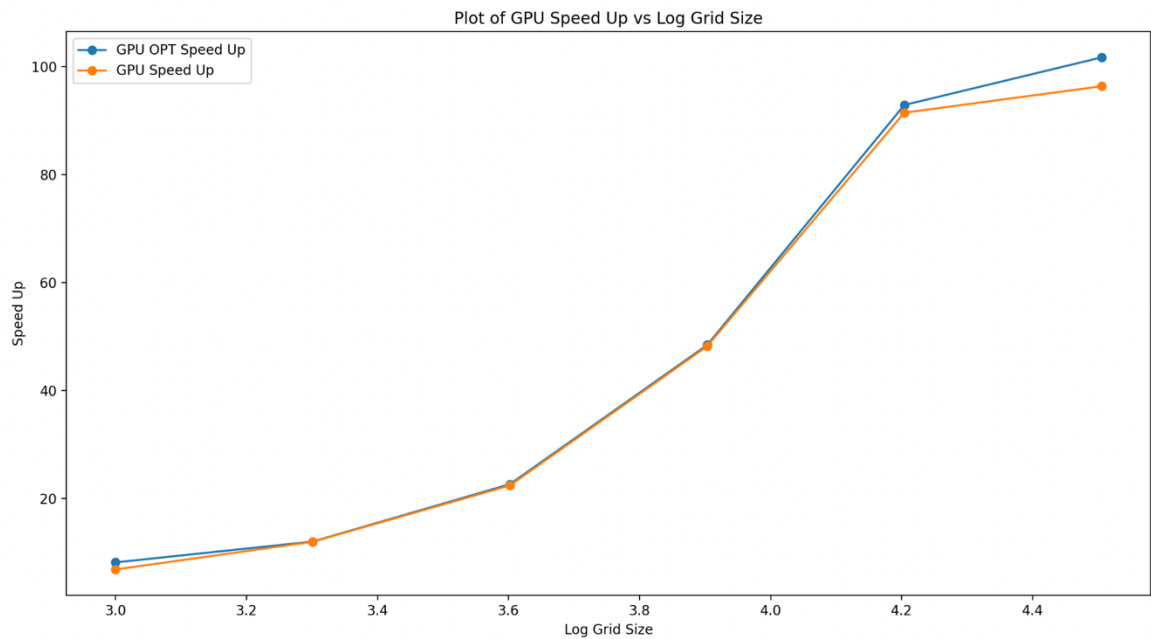


Figure 2: Plot of GPU Speedup vs Log Grid Size for optimized and non-optimized code for Matrix Vector Product. We see the optimized code have better performance at the high end of grid size ($n = 32,000$)

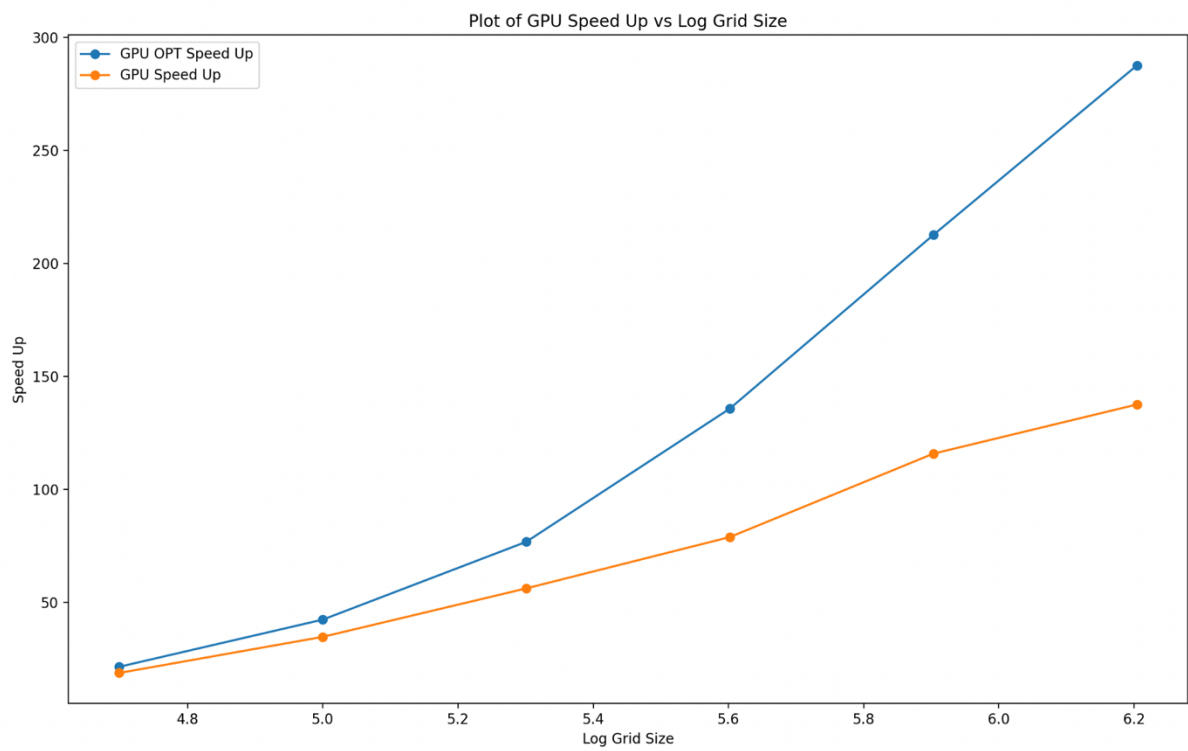


Figure 3: Plot of GPU Speedup vs Log Grid Size for optimized and non-optimized code Matrix Free Matrix Vector Product. We see the Opt version of the code outperforms for all grid sizes up to ($n = 1,600,000$).

Figures of Speedup for CG:

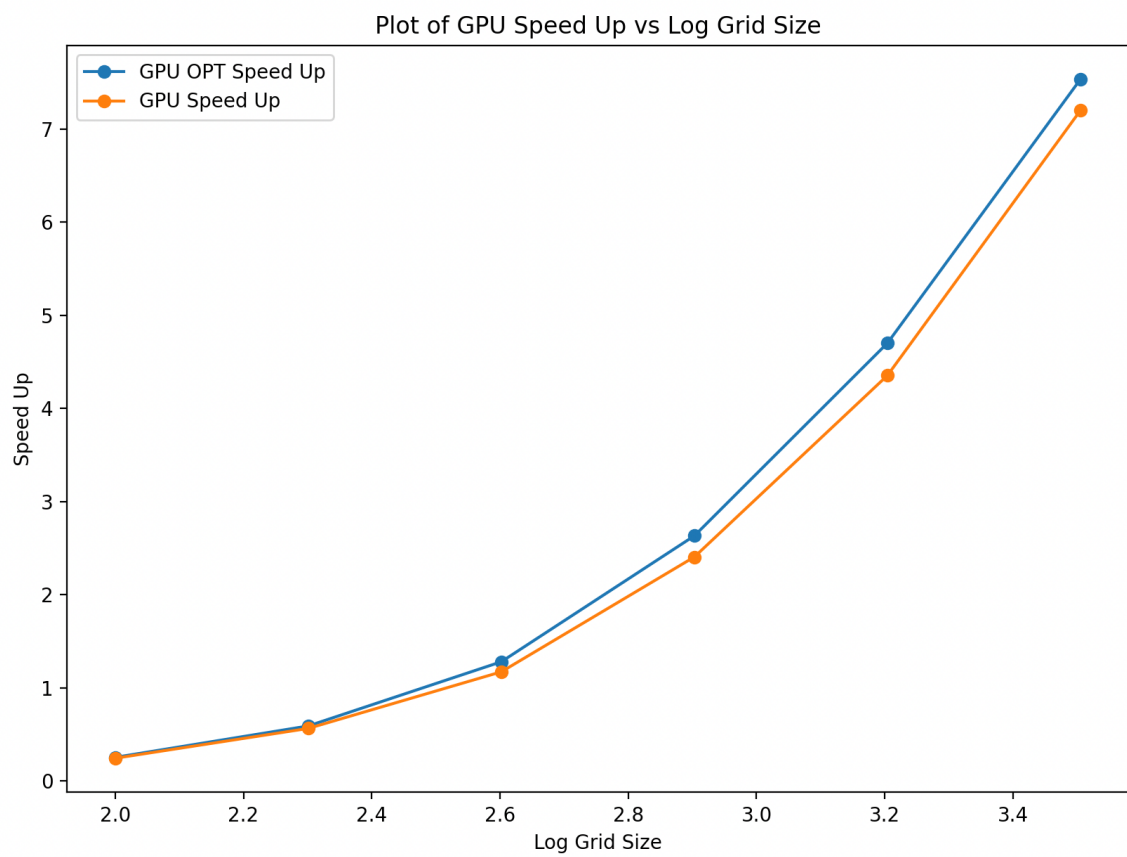


Figure 4: Plot of GPU Speedup vs Log Grid Size for optimized and non-optimized code Dense CG. We see the GPU outperform as the problem size gets bigger, but the CPU is better at small grid sizes.

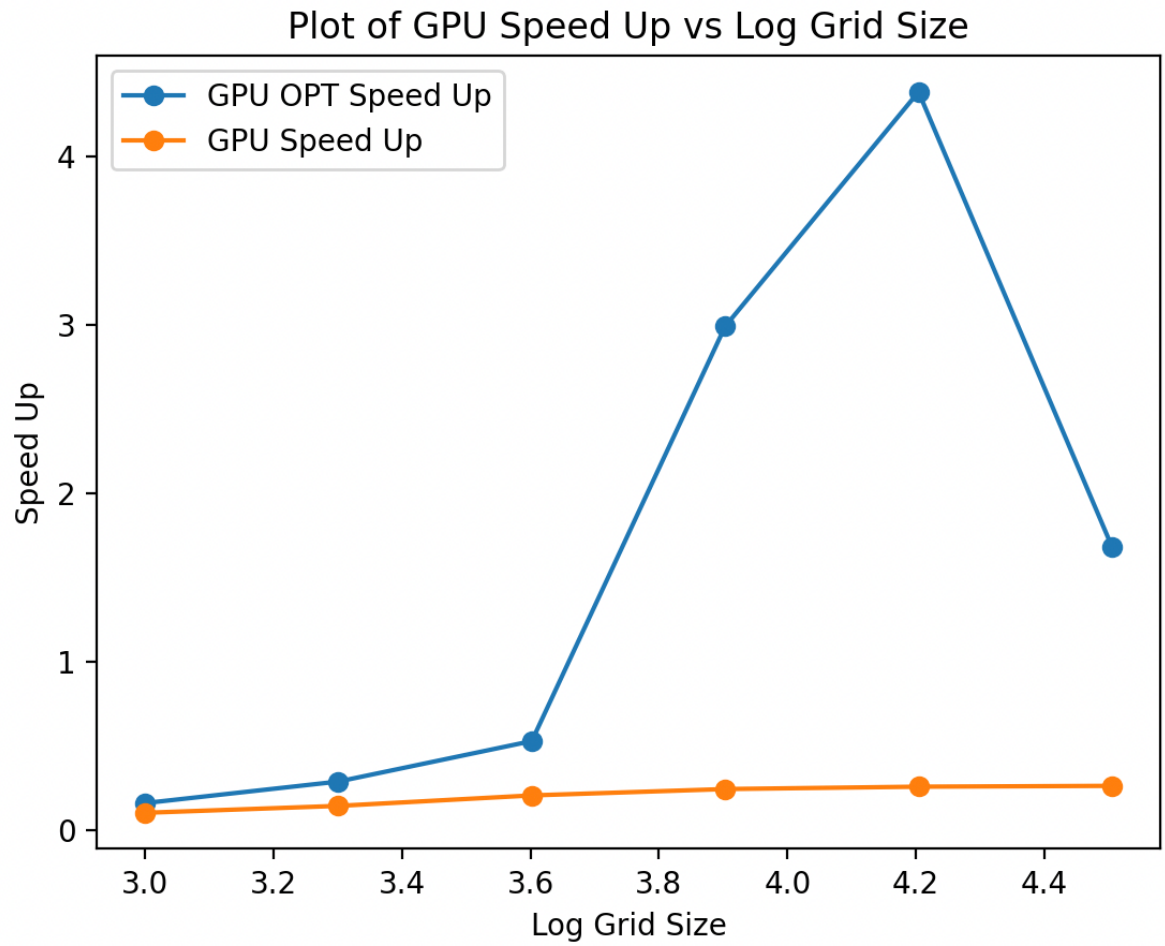


Figure 5: Plot of GPU Speedup vs Log Grid Size for optimized and non-optimized code Matrix Free CG. We see the GPU outperform as the problem size gets bigger, but the CPU is better at small grid sizes.